AE2

Ciro Ferreira

Parte 1: Descripción del Problema y Justificación de Estructuras

Descripción del Problema: El objetivo es crear un programa que permita digitalizar y llevar el control de las ventas semanales del buffet de la facultad. El sistema debe tener un catálogo de productos predefinido y permitirle al usuario registrar transacciones ingresando el día de la semana (1 al 5), el código del producto y la cantidad. Con esta información, el algoritmo debe acumular las unidades vendidas por cada producto y calcular la recaudación de cada día, permitiendo generar un reporte general sin que los datos se borren al cerrar la aplicación.

Estructuras de Datos Seleccionadas: Para resolver este problema con los conceptos vistos en la Unidad 1, utilicé las siguientes estructuras:

- **Arreglos unidimensionales (Listas paralelas):** Para el catálogo y los acumuladores de ventas, decidí utilizar listas paralelas en lugar de una matriz bidimensional. Esta estructura es ideal para este caso porque me permite separar los tipos de datos (una lista para textos como los nombres, y otras listas para números como precios y cantidades), vinculándolos de forma sencilla a través del mismo número de índice.
- **Archivo de texto secuencial:** Como las variables en los arreglos son volátiles y se limpian al finalizar la ejecución, implementé la lectura y escritura de un archivo de texto común (ventas.txt). Al iniciar, el programa vuelca los datos del archivo en los arreglos; y al salir, sobrescribe el archivo con los totales actualizados, logrando la persistencia de los datos requerida.

Diseño Lógico y Pseudocódigo (PSeInt)

Especificación del manejo de datos:

- **Carga inicial:** El algoritmo comienza definiendo la dimensión de los arreglos. Se inicializan manualmente los códigos, nombres y precios del catálogo. Luego, mediante un ciclo *Para*, se ponen en cero todos los contadores de las ventas y la recaudación diaria (o se cargan desde el archivo si ya existiera).
- **Procesamiento:** El flujo principal está controlado por un menú dentro de un ciclo *Mientras*. Al registrar una transacción, se solicita el código del producto. El programa utiliza un ciclo *Para* para recorrer el arreglo de códigos; si encuentra una coincidencia, guarda ese índice. Luego, mediante estructuras condicionales (*Si-Entonces*), verifica que el índice sea válido y que el día esté entre 1 y 5. Si todo es correcto, utiliza el índice

encontrado para sumar la cantidad y calcular el total de plata de ese día.

- **Recuperación:** En la opción de informes, el sistema simplemente itera sobre las listas paralelas para mostrar los valores acumulados. Al finalizar (opción Salir), se graban los datos en el disco.

```
Algoritmo Buffet_AE2
    Definir codigos, precios, cantidades, dia, cod, cant, i, opc, indice Como Entero
    Definir nombres Como Cadena
    Definir recaudacion_dias, total_semana Como Real

    Dimension codigos[6], nombres[6], precios[6], cantidades[6], recaudacion_dias[5]

    codigos[1] ← 101; nombres[1] ← "Cafe con leche"; precios[1] ← 1500; cantidades[1] ← 0
    codigos[2] ← 102; nombres[2] ← "Medialuna"; precios[2] ← 700; cantidades[2] ← 0
    codigos[3] ← 103; nombres[3] ← "Sandwich de miga"; precios[3] ← 1800; cantidades[3] ← 0
    codigos[4] ← 104; nombres[4] ← "Agua mineral 500 ml"; precios[4] ← 1000; cantidades[4] ← 0
    codigos[5] ← 105; nombres[5] ← "Empanada"; precios[5] ← 1200; cantidades[5] ← 0
    codigos[6] ← 106; nombres[6] ← "Chipa (100 g)"; precios[6] ← 1300; cantidades[6] ← 0

    Para i ← 1 Hasta 5 Con Paso 1 Hacer
        recaudacion_dias[i] ← 0
    FinPara

    opc ← 0
    Mientras opc ≠ 3 Hacer
        Escribir "1. Registrar venta | 2. Ver informes | 3. Guardar y salir"
        Leer opc

        Segun opc Hacer
            1:
                Escribir "Dia (1 a 5): "; Leer dia
                Escribir "Codigo: "; Leer cod
                Escribir "Cantidad: "; Leer cant

                indice ← 0
                Para i ← 1 Hasta 6 Con Paso 1 Hacer
                    Si codigos[i] = cod Entonces
                        indice ← i
                    FinSi
                FinPara

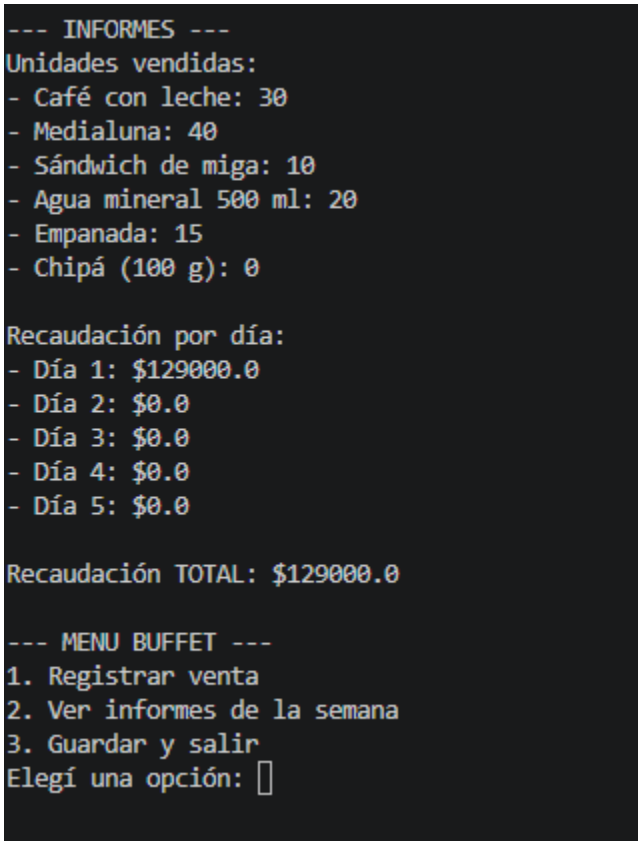
                Si indice > 0 Y dia ≥ 1 Y dia ≤ 5 Y cant > 0 Entonces
                    cantidades[indice] ← cantidades[indice] + cant
                    recaudacion_dias[dia] ← recaudacion_dias[dia] + (cant * precios[indice])
```

```
40                Escribir "Venta guardada."
41            Sino
42                Escribir "Error de datos."
43            FinSi
44        2:
45            total_semana ← 0
46            Para i ← 1 Hasta 6 Con Paso 1 Hacer
47                Escribir nombres[i], ": ", cantidades[i]
48            FinPara
49            Para i ← 1 Hasta 5 Con Paso 1 Hacer
50                Escribir "Dia ", i, ": $", recaudacion_dias[i]
51                total_semana ← total_semana + recaudacion_dias[i]
52            FinPara
53            Escribir "Total: $", total_semana
54        3:
55            Escribir "Guardando y saliendo..."
56    FinSegun
57  FinMientras
58  FinAlgoritmo
```

```
Día (1 Lunes - 5 Viernes): 8
Código del producto: 999
Cantidad vendida: 12
Error: Datos incorrectos.
```

```
Recaudación por día:
- Día 1: $15000.0
- Día 2: $0.0
- Día 3: $0.0
- Día 4: $0.0
- Día 5: $0.0

Recaudación TOTAL: $15000.0
```



- **Parte 5: Reflexión sobre estructuras de datos**

Durante la fase de diseño, la principal alternativa que evalué y descarté fue utilizar una única matriz bidimensional para almacenar tanto el catálogo como las ventas. Descarté esta opción porque las matrices estándar dificultan la combinación de distintos tipos de datos (como el nombre en texto y el precio en número) sin generar conflictos lógicos o requerir conversiones constantes.

Al optar por los arreglos unidimensionales paralelos, aprendí que es una estructura muy eficiente y fácil de comprender para resolver asociaciones básicas, ya que todo se conecta mediante una variable iteradora (i). Sin embargo, también noté su principal limitación: la falta de flexibilidad. Si en el futuro el buffet necesitara ampliar su catálogo con diez productos nuevos, sería necesario modificar manualmente el código fuente para aumentar la longitud de todas las listas en paralelo y no desfasar la información, lo cual restaría agilidad al mantenimiento del sistema

Portafolio:

1. Selección de estructuras

Para guardar el catálogo de productos y las ventas de la semana elegí listas paralelas (codigos, nombres, precios y cantidades) y una lista de 5 posiciones para la recaudación de cada día. Las elegí porque cada dato es de un tipo distinto (texto y números) y se vinculan con el mismo índice. Además usé un archivo de texto para que los datos no se pierdan al cerrar el programa.

2.Borrador de diseño

```
Algoritmo Buffet_AE2
    Definir codigos, precios, cantidades, dia, cod, cant, i, opc, indice Como Entero
    Definir nombres Como Cadena
    Definir recaudacion_dias, total_semana Como Real

    Dimension codigos[6], nombres[6], precios[6], cantidades[6], recaudacion_dias[5]

    codigos[1] ← 101; nombres[1] ← "Cafe con leche"; precios[1] ← 1500; cantidades[1] ← 0
    codigos[2] ← 102; nombres[2] ← "Medialuna"; precios[2] ← 700; cantidades[2] ← 0
    codigos[3] ← 103; nombres[3] ← "Sandwich de miga"; precios[3] ← 1800; cantidades[3] ← 0
    codigos[4] ← 104; nombres[4] ← "Agua mineral 500 ml"; precios[4] ← 1000; cantidades[4] ← 0
    codigos[5] ← 105; nombres[5] ← "Empanada"; precios[5] ← 1200; cantidades[5] ← 0
    codigos[6] ← 106; nombres[6] ← "Chipa (100 g)"; precios[6] ← 1300; cantidades[6] ← 0

    Para i ← 1 Hasta 5 Con Paso 1 Hacer
        recaudacion_dias[i] ← 0
    FinPara

    opc ← 0
    Mientras opc ≠ 3 Hacer
        Escribir "1. Registrar venta | 2. Ver informes | 3. Guardar y salir"
        Leer opc

        Segun opc Hacer
            1:
                Escribir "Dia (1 a 5): "; Leer dia
                Escribir "Codigo: "; Leer cod
                Escribir "Cantidad: "; Leer cant

                indice ← 0
                Para i ← 1 Hasta 6 Con Paso 1 Hacer
                    Si codigos[i] = cod Entonces
                        indice ← i
                    FinSi
                FinPara

                Si indice > 0 Y dia ≥ 1 Y dia ≤ 5 Y cant > 0 Entonces
                    cantidades[indice] ← cantidades[indice] + cant
                    recaudacion_dias[dia] ← recaudacion_dias[dia] + (cant * precios[indice])
```

```
                Escribir "Venta guardada."
            Sino
                Escribir "Error de datos."
            FinSi
        2:
            total_semana ← 0
            Para i ← 1 Hasta 6 Con Paso 1 Hacer
                Escribir nombres[i], ": ", cantidades[i]
            FinPara
            Para i ← 1 Hasta 5 Con Paso 1 Hacer
                Escribir "Dia ", i, ": $", recaudacion_dias[i]
                total_semana ← total_semana + recaudacion_dias[i]
            FinPara
            Escribir "Total: $", total_semana
        3:
            Escribir "Guardando y saliendo..."
    FinSegun
  FinMientras
FinAlgoritmo
```

3. Primer avance de código

```python
codigos = []
nombres = []
precios = []
cantidades_vendidas = []

datos_nuevos = [
    [101, "Café con leche", 1500],
    [102, "Medialuna", 700],
    [103, "Sándwich de miga", 1800],
    [104, "Agua mineral 500 ml", 1000],
    [105, "Empanada", 1200],
    [106, "Chipá (100 g)", 1300]
]

for producto in datos_nuevos:
    cod = producto[0]
    repetido = False
    for c in codigos:
        if c == cod:
            repetido = True

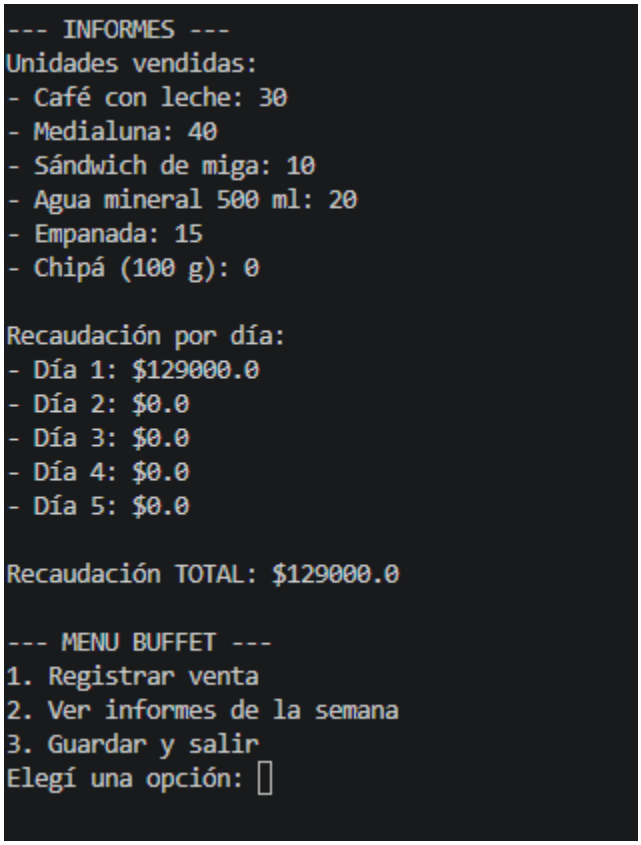
    if repetido == False:
        codigos.append(cod)
        nombres.append(producto[1])
        precios.append(producto[2])
        cantidades_vendidas.append(0)
    else:
        print("Código repetido:", cod)

for i in range(len(codigos)):
    print(codigos[i], nombres[i], precios[i])
```

4.Registro de pruebas: cargar las ventas del lunes del cuaderno en el día 1 (30 cafés con leche, 40 medialunas, 10 sándwiches de miga, 20 aguas, 15 empanadas).

Resultado esperado: recaudación del día 1 de $129000.0 (45000 + 28000 + 18000 + 20000 + 18000).

Resultado obtenido: Día 1: $129000.0.



Pruebas:

Prueba 1 (Caso límite - Archivo inexistente):

- **Dato de entrada:** Ejecutar el programa por primera vez asegurando que el archivo ventas.txt no exista en el directorio.
- **Resultado esperado:** El sistema debe omitir el paso de lectura sin arrojar errores de compilación, inicializar todos los arreglos en cero, y crear el archivo recién al momento de cerrar el programa.
- **Resultado obtenido:** El programa inició correctamente en cero y generó el archivo de texto al elegir la opción de salir.

Prueba 2 (Caso normal - Acumulación de ventas):

- **Dato de entrada:** Ingresar una venta para el día Lunes (día 1) del producto con código

101, cantidad: 10.

- **Resultado esperado:** Al solicitar el reporte, el programa debe mostrar 10 unidades vendidas del producto correspondiente y reflejar el cálculo exacto de dinero (precio x 10) en la recaudación del día 1.

```
--- INFORMES ---
Unidades vendidas:
- Café con leche: 10
- Medialuna: 0
- Sándwich de miga: 0
- Agua mineral 500 ml: 0
- Empanada: 0
- Chipá (100 g): 0

Recaudación por día:
- Día 1: $15000.0
- Día 2: $0.0
- Día 3: $0.0
- Día 4: $0.0
- Día 5: $0.0

Recaudación TOTAL: $15000.0
```

- **Resultado obtenido:**

Prueba 3 (Caso límite - Validación de entrada):

- **Dato de entrada:** Intentar registrar una venta en un día inexistente (ejemplo: día 8) o introducir un código de producto que no está en las listas (ejemplo: código 999).

- **Resultado esperado:** Las estructuras condicionales deben capturar el error, impedir que se sumen datos a índices inexistentes y mostrar un mensaje de alerta al usuario.

- **Resultado obtenido:**

```
Día (1 Lunes - 5 Viernes): 8
Código del producto: 999
Cantidad vendida: 12
Error: Datos incorrectos.
```

Link del repositorio : https://github.com/CiroFerreira-lab/Programacion-I.git